综合练习

人员

王承周、阮文璋 到课

上周作业检查

上上周作业链接: https://vjudge.net/contest/810246

Rank	Team	Score	Penalty	A 4 / 8	B 8 / 9	C 7 / 17	D 2 / 2	E 4 / 18
1	☆ r111	5	95255	13:12:49:00	12:05:36:00	13:11:49:00 (-1)	**12:06:51:00**	**14:13:30:00 (-2)**
2	☆ wcz123bc	5	122080	18:02:42:00 (-1)	12:06:01:53	18:02:27:00	18:03:37:00	18:03:13:00 (-1)
3	☆ junyan007008	4	76106	**12:01:38:00 (-1)**	**12:00:45:00**	12:01:07:00 (-3)		16:13:16:00 (-7)
4	☆ lijinshu	4	88182	18:11:38:00 (-1)	12:00:48:00	12:01:03:00 (-2)		18:13:53:00 (-4)
5	☆ chenxinmiao (陈欣妙)	2	34651		12:00:46:00	12:00:45:46		
6	☆ yyycj	2	34661	(-1)	12:00:57:33	**12:00:44:15**		
7	☆ wyb1111 (wwyb)	2	35411		12:05:43:00 (-1)	12:06:48:00 (-4)		
8	☆ chujinxuan (褚锦轩)	1	17326		12:00:46:22			

上周作业链接: https://vjudge.net/contest/816673

本周作业

https://vjudge.net/contest/818166 (课上讲了上周的 A ~ E 题, 课后作业是本周的 A ~ E 题)

课堂表现

今天的题目整体来说还是难度稍大的, 同学们课上过题不多, 需要同学们课下好好复盘一下, 把题目补出来。

课堂内容

CF1379C Choosing flowers

枚举最后只选哪个物品的 b 结尾, 然后选那些 a 比他大的物品即可 (用 二分 + 前缀和 维护即可)

```cpp
#include <bits/stdc++.h>
#define int long long

using namespace std;
```

```
const int maxn = 1e5 + 5;
struct node {
  int a, b;
  bool operator < (const node& p) const { return a < p.a; }
} w[maxn];
int c[maxn], p[maxn];

int get_sum(int l, int r) { return p[r] - p[l-1]; }

void solve() {
  int n, m; cin >> n >> m;
  for (int i = 1; i <= m; ++i) cin >> w[i].a >> w[i].b;
  sort(w+1, w+m+1);
  for (int i = 1; i <= m; ++i) c[i] = w[i].a, p[i] = p[i-1] + c[i];

  int res = 0;
  for (int i = 1; i <= m; ++i) {
    int val = w[i].b;
    int pos = upper_bound(c+1, c+m+1, val) - c;

    if (m-pos+1 >= n) res = max(res, get_sum(m-n+1, m));
    else {
      if (pos > i) res = max(res, get_sum(pos,m) + w[i].a + val*(n-m+pos-2));
      else res = max(res, get_sum(pos,m) + val*(n-m+pos-1));
    }
  }
  cout << res << endl << endl;
}

signed main()
{
  int T; cin >> T;
  while (T -- ) solve();
  return 0;
}
```

CF1904D1 Set To Max

对于第 i 个物品来说, 如果 a[i] < b[i], 那么就往左或往右找到一个和 b[i] 相同的 a 值

想更改过来的条件是: 要满足区间没有 a 值比这个 a 值小, 没有 b 值比这个 a 值大

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 1000 + 5;
int a[maxn], b[maxn];

void solve() {
  int n; cin >> n;
```

```
  for (int i = 1; i <= n; ++i) cin >> a[i];
  for (int i = 1; i <= n; ++i) cin >> b[i];

  for (int i = 1; i <= n; ++i) {
    if (a[i] > b[i]) { cout << "NO" << endl; return; }
    if (a[i] == b[i]) continue;

    int l = i, r = i;
    while (true) {
      --l;
      if (l == 0) { l = -1; break; }
      if (a[l]>b[i] || b[l]<b[i]) { l = -1; break; }
      if (a[l] == b[i]) break;
    }
    while (true) {
      ++r;
      if (r > n) { r = -1; break; }
      if (a[r]>b[i] || b[r]<b[i]) { r = -1; break; }
      if (a[r] == b[i]) break;
    }

    if (l==-1 && r==-1) { cout << "NO" << endl; return; }
  }

  cout << "YES" << endl;
}

int main()
{
  int T; cin >> T;
  while (T -- ) solve();
  return 0;
}
```

CF1904D2 Set To Max

思路跟上面题一样, 就是找区间最大值、区间最小值的过程, 可以用 st 表实现; 往左往右找相同 a 值的过程, 可以用 vector数组+二分 实现

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e5 + 5, M = 20;
int a[maxn], b[maxn];
int _lg2[maxn], fa[maxn][M], fb[maxn][M];
vector<int> vec[maxn];

int get_Amax(int l, int r) {
  int k = _lg2[r-l+1];
  return max(fa[l][k], fa[r-(1<<k)+1][k]);
```

```
}
int get_Bmin(int l, int r) {
  int k = _lg2[r-l+1];
  return min(fb[l][k], fb[r-(1<<k)+1][k]);
}

void solve() {
  int n; cin >> n;
  for (int i = 1; i <= n; ++i) cin >> a[i], fa[i][0] = a[i];
  for (int i = 1; i <= n; ++i) cin >> b[i], fb[i][0] = b[i];

  for (int k = 1; k < M; ++k) {
    for (int i = 1; i+(1<<k)-1 <= n; ++i) {
      fa[i][k] = max(fa[i][k-1], fa[i+(1<<(k-1))][k-1]);
      fb[i][k] = min(fb[i][k-1], fb[i+(1<<(k-1))][k-1]);
    }
  }

  for (int i = 1; i <= n; ++i) vec[i].clear();
  for (int i = 1; i <= n; ++i) vec[a[i]].push_back(i);
  for (int i = 1; i <= n; ++i) {
    vec[i].push_back(0), vec[i].push_back(n+1);
    sort(vec[i].begin(), vec[i].end());
  }

  for (int i = 1; i <= n; ++i) {
    if (a[i] > b[i]) { cout << "NO" << endl; return; }
    if (a[i] == b[i]) continue;

    int l = upper_bound(vec[b[i]].begin(), vec[b[i]].end(), i) - vec[b[i]].begin()
- 1;
    int r = upper_bound(vec[b[i]].begin(), vec[b[i]].end(), i) -
vec[b[i]].begin();
    l = vec[b[i]][l], r = vec[b[i]][r];

    if (l!=0 && get_Amax(l,i)<=b[i] && get_Bmin(l,i)>=b[i]) continue;
    if (r!=n+1 && get_Amax(i,r)<=b[i] && get_Bmin(i,r)>=b[i]) continue;

    cout << "NO" << endl;
    return;
  }

  cout << "YES" << endl;
}

int main()
{
  for (int i = 0; (1<<i) < maxn; ++i) _lg2[1<<i] = i;
  for (int i = 1; i < maxn; ++i) {
    if (!_lg2[i]) _lg2[i] = _lg2[i-1];
  }

  int T; cin >> T;
  while (T -- ) solve();
```

```
  return 0;
}
```

CF1905D Cyclic MEX

可以用 线段树上二分+线段树区间修改+线段树区间查询 维护

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 1e6 + 5;
struct node {
  int l, r, flag, sum, maxx;
} tr[maxn*2*4];
int w[maxn], suf_min[maxn];

void pushup(int u) {
  tr[u].sum = tr[u*2].sum + tr[u*2+1].sum;
  tr[u].maxx = max(tr[u*2].maxx, tr[u*2+1].maxx);
}
void pushdown(int u) {
  if (tr[u].flag != -1) {
    node &uu = tr[u], &ll = tr[u*2], &rr = tr[u*2+1];
    ll.flag = uu.flag, ll.maxx = uu.flag, ll.sum = (ll.r-ll.l+1) * ll.flag;
    rr.flag = uu.flag, rr.maxx = uu.flag, rr.sum = (rr.r-rr.l+1) * rr.flag;
    uu.flag = -1;
  }
}
void build(int u, int l, int r) {
  tr[u] = {l, r, -1, 0, 0};
  if (l == r) return;

  int mid = (l + r) / 2;
  build(u*2, l, mid), build(u*2+1, mid+1, r);
}
void modify(int u, int l, int r, int k) {
  if (tr[u].l>=l && tr[u].r<=r) {
    tr[u].flag = k, tr[u].maxx = k, tr[u].sum = (tr[u].r - tr[u].l + 1) * k;
    return;
  }

  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2;
  if (l <= mid) modify(u*2, l, r, k);
  if (r > mid) modify(u*2+1, l, r, k);
  pushup(u);
}
int query(int u, int l, int r) {
  if (tr[u].l>=l && tr[u].r<=r) return tr[u].sum;
```

```
  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2, res = 0;
  if (l <= mid) res += query(u*2, l, r);
  if (r > mid) res += query(u*2+1, l, r);
  return res;
}

int queryPos(int u, int l, int r, int c) {  // 区间 [l,r] 中第一个 >w[i] 的位置
  if (tr[u].l == tr[u].r) return tr[u].l;

  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2;
  if (r <= mid) return queryPos(u*2, l, r, c);
  if (l > mid) return queryPos(u*2+1, l, r, c);

  if (tr[u*2].maxx > c) return queryPos(u*2, l, r, c);
  return queryPos(u*2+1, l, r, c);
}

void solve() {
  int n; cin >> n;
  for (int i = 1; i <= n; ++i) cin >> w[i];
  suf_min[n] = n;
  for (int i = n-1; i >= 1; --i) suf_min[i] = min(suf_min[i+1], w[i+1]);

  build(1, 1, 2*n);
  for (int i = 1; i <= n; ++i) modify(1, i, i, suf_min[i]);
  int res = query(1, 1, n);

  for (int i = 1, j = n+1; i <= n-1; ++i, ++j) {
    int pos = queryPos(1, i, j-1, w[i]);  // 区间 (i, j-1) 中第一个 >w[i] 的位置
    modify(1, pos, j-1, w[i]); modify(1, j, j, n);
    res = max(res, query(1, i+1, j));
  }
  cout << res << "\n";
}

signed main()
{
  ios::sync_with_stdio(false);
  cin.tie(0);

  int T; cin >> T;
  while (T -- ) solve();
  return 0;
}
```

CF479E Riding in a Lift

设 f[i][j]: 移动 i 次后, 到达第 j 个位置有多少方案

那么 f[i][j] 是由 f[i-1][l] + ... + f[i-1][r] - f[i-1][j] 得来的, 可以用前缀和实现 O(1) 转移

l 和 r 这两个边界也可以根据题意 O(1) 求

```cpp
#include <bits/stdc++.h>

using namespace std;

const int maxn = 5000 + 5;
const int mod = 1e9 + 7;
int f[maxn][maxn], p[maxn][maxn];
int n, a, b, K;

int get_sum(int id, int l, int r) {
  l = max(l, 1), r = min(r, n);
  return (p[id][r] - p[id][l-1] + mod) % mod;
}

int main()
{
  cin >> n >> a >> b >> K;
  f[0][a] = 1;
  for (int i = 1; i <= n; ++i) p[0][i] = p[0][i-1] + f[0][i];

  for (int i = 1; i <= K; ++i) {
    for (int j = 1; j <= n; ++j) {
      if (a<b && j<b) {
        int l = 1, r = (j + b) / 2;
        while (r-j >= b-r) --r;
        f[i][j] = (get_sum(i-1, l, r) - f[i-1][j] + mod) % mod;
      }

      if (a>b && j>b) {
        int l = (b + j) / 2, r = n;
        while (j-l >= l-b) ++l;
        f[i][j] = (get_sum(i-1, l, r) - f[i-1][j] + mod) % mod;
      }
    }
    for (int j = 1; j <= n; ++j) p[i][j] = (p[i][j-1] + f[i][j]) % mod;
  }

  cout << p[K][n] << endl;
  return 0;
}
```